

MALAR: Multi-scale Adaptive Learning with Abstraction and Recall

A general-purpose temporal learning model over scale-dependent graphs with topological memory

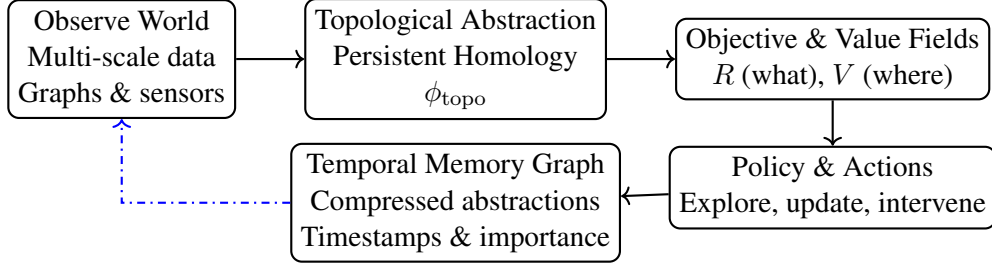
immediate

September 13, 2026

Abstract

MALAR is a general-purpose learning framework for real-world systems whose observations are multidimensional, scale-dependent, and temporally evolving. MALAR represents the world as a collection of coupled graphs and higher-order complexes, computes objective and value fields that drive automated exploration and intervention, and compresses learned structure into a time-stamped memory graph. Instead of storing raw trajectories, MALAR stores compact graph and topological signatures (including persistent-homology summaries) together with importance weights and learned timestamps. Memory entries are updated when novelty, change, or predicted goal gain exceeds a threshold, and are retrieved by similarity plus temporal gating for both forward decision support and reverse generation of plausible world states. We provide a unified mathematical formulation, an all-in-one algorithm, comparisons to existing continual learning and memory-augmented models, and applied the suitability of MALAR for example use cases but not limited to these use cases: (i) multi-modal monitoring and screening for infectious disease in airport arrival environments using biological sensor arrays (ii) a general industrial digital-twin setting (illustrative) where multi-scale sensor graphs and topology drive closed-loop optimization.

Keywords. continual learning; memory-augmented learning; topological data analysis; persistent homology; graph neural networks; active learning; temporal representation; anomaly detection.



Graphical Abstract. MALAR is a general-purpose learning system that observes multi-scale worlds and prioritizes learning via objective and value fields, acts autonomously, and stores compact temporal memory for recall

1 Introduction

Many real-world learning problems are not independent and identically distributed datasets but evolving systems: sensors drift, new regimes appear, operating conditions change, and the scale at which structure matters may shift over time. Standard deep learning pipelines store parameters but not the structured knowledge needed to detect regime shifts, compare with historical patterns, and recover or simulate plausible prior states. MALAR addresses this by combining (a) a multi-scale structured world representation; (b) distributed objective and value fields for prioritization; and (c) an explicit temporal memory graph storing compressed topological and graph features with learned timestamps and importance weights.

Contributions.

- A unified, domain-independent formulation for learning over multi-scale graphs and complexes.
- Time-stamped topological memory: compressed graph/persistent-homology features stored and updated under novelty/goal-gain triggers.
- A single all-in-one algorithm integrating observation, abstraction, prioritization, action selection, memory update, and recall/generation.
- Comparison table vs. major families of memory-augmented and continual learning frameworks.
- Two application sections including airport bio-sensing.

2 Related Work

MALAR intersects continual learning and replay, memory-augmented neural networks, dynamic graph learning, active learning, and topological data analysis (TDA). MALAR differs in its explicit combination of objective and value fields defined on a structured world, plus a compressed time-aware memory graph that stores topological signatures rather than raw samples, enabling both similarity-based recall and reverse world-state generation.

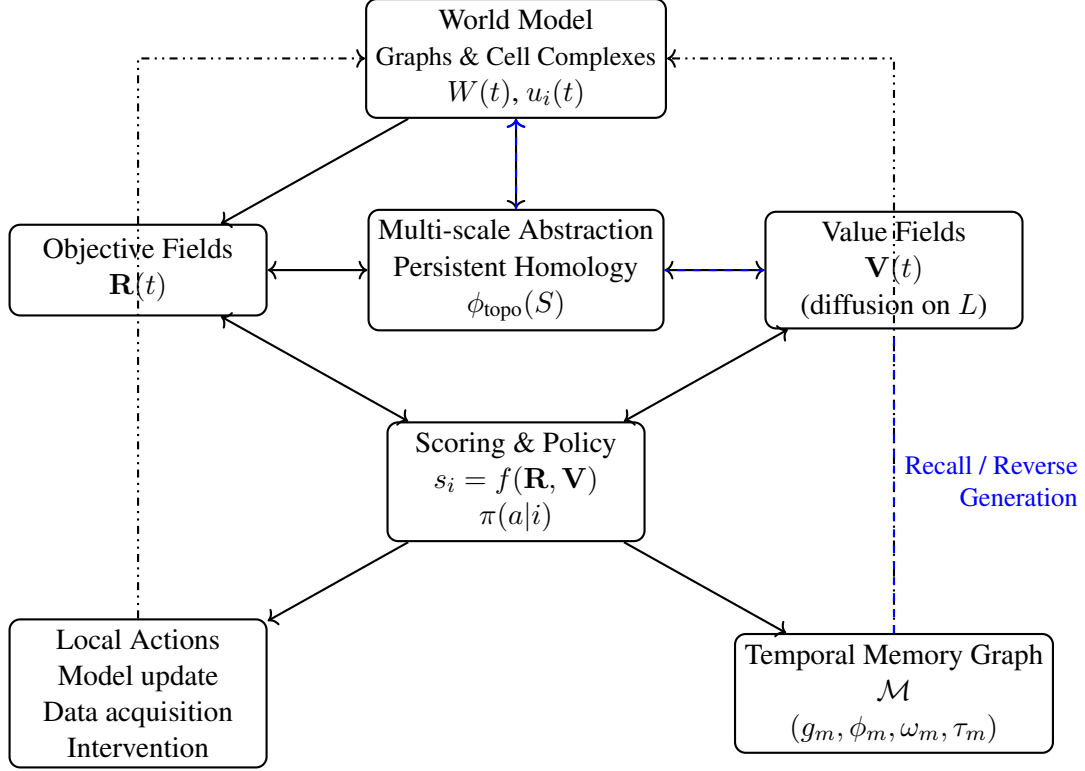


Figure 1: MALAR: Multi-scale Adaptive Learning with Abstraction and Recall. The system observes a structured world, extracts multi-scale topological abstractions, computes objective and value fields, selects actions under resource constraints, and stores compressed knowledge in a temporal memory graph enabling recall and reverse generation.

3 The MALAR Model

3.1 World as multi-scale temporal graphs and complexes

Let the world at time t be represented as a structured object

$$W(t) = (I, E(t), C(t), L(t)), \quad u_i(t) \in \mathbb{R}^{d_u}, \quad (1)$$

where I indexes locations/entities, $E(t)$ are edges, $C(t)$ are higher-order cells, and $L(t)$ denotes diffusion operators (graph Laplacians and/or Hodge Laplacians).

3.2 Objective and value fields

MALAR maintains K objective fields and M value fields:

$$R_i(t) = (R_i^{(1)}(t), \dots, R_i^{(K)}(t)), \quad V_i(t) = (V_i^{(1)}(t), \dots, V_i^{(M)}(t)). \quad (2)$$

Values propagate across structure via diffusion with sources:

$$V(t + \Delta t) = V(t) - \Delta t L(t) V(t) + \Delta t S_V(u(t), R(t)). \quad (3)$$

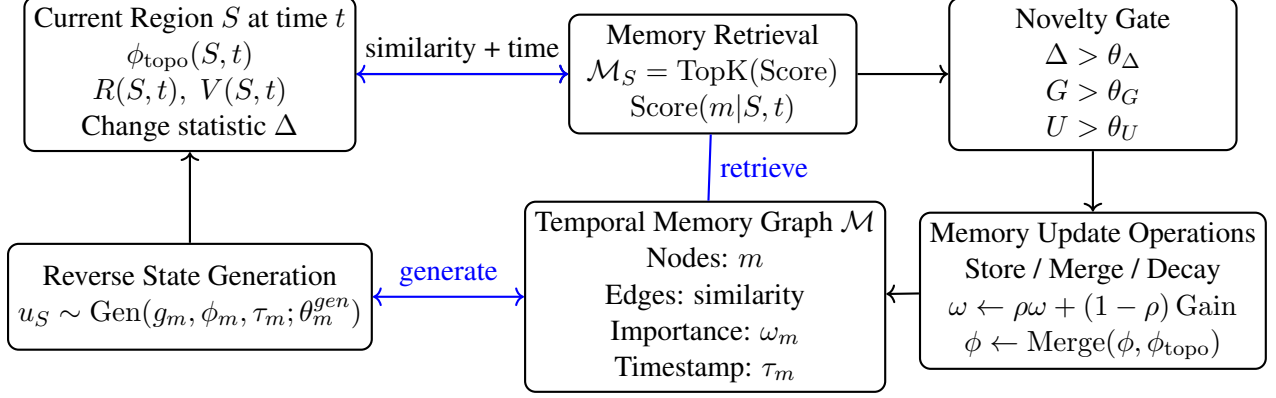


Figure 2: **Memory-centric MALAR architecture.** Current multi-scale abstractions are compared against a temporal memory graph using similarity and time-gated retrieval. Novelty, uncertainty, or goal gain triggers memory insertion or merge; otherwise entries decay. Stored memory supports both forward decision support and reverse world-state generation.

3.3 Topological abstraction and compression

For regions S (subgraphs/subcomplexes) and layers $r \in \{1, \dots, R\}$, MALAR builds filtrations and computes persistence diagrams $D_{r,S}^{(d)}$. These are embedded into vectors $\phi_r(S)$ and concatenated:

$$\phi_{\text{topo}}(S) = [\phi_1(S) \parallel \phi_2(S) \parallel \dots \parallel \phi_R(S)]. \quad (4)$$

3.4 Temporal memory graph with learned timestamps

MALAR stores compressed information in a temporal memory graph $\mathcal{M}$. Each memory item m stores:

$$m = (g_m, \phi_m, \omega_m, \tau_m, \theta_m^{\text{gen}}), \quad (5)$$

where g_m is a compact graph/complex abstraction, ϕ_m is a topological descriptor, ω_m is importance, τ_m is a learned real-world timestamp, and θ_m^{gen} parameterizes optional reverse generation.

4 Learning, Recall, and Generation

4.1 Comparing current state to memory

For region S at time t , MALAR retrieves memory by similarity and temporal gating:

$$\text{Score}(m \mid S, t) = \text{Sim}(\phi_{\text{topo}}(S, t), \phi_m) + \lambda \text{TimeGate}(t, \tau_m) + \mu \omega_m. \quad (6)$$

Low score or high change indicates novelty and triggers insertion/merge.

4.2 Memory update rule

Let Δ be a change statistic (e.g. PH distance, embedding distance) and G predicted goal gain. Store if:

$$\Delta > \theta_\Delta \quad \text{or} \quad G > \theta_G \quad \text{or} \quad \text{Uncertainty}(S) > \theta_U. \quad (7)$$

Otherwise update via importance-weighted merging/decay:

$$\omega_m \leftarrow \rho \omega_m + (1 - \rho) \text{Gain}(S), \quad \phi_m \leftarrow \text{Merge}(\phi_m, \phi_{\text{topo}}(S)). \quad (8)$$

Algorithm 1 MALAR (Multi-scale Adaptive Learning with Abstraction and Recall)

```
1: Input: initial world  $W(0)$ , states  $u(0)$ , memory  $\mathcal{M} \leftarrow \emptyset$ , budgets  $B$ , thresholds  $\Theta$ 
2: for  $t = 0, 1, 2, \dots$  do
3:   Observe data;  $W(t) \leftarrow \text{UPDATEWORLD}(W(t), \text{data}_t)$ 
4:    $u(t) \leftarrow F_{\text{rep}}(u(t), W(t))$ 
5:   for all regions  $S \leftarrow \text{REGIONSAMPLER}(W(t), B)$  do
6:      $\phi_{\text{topo}}(S, t) \leftarrow \text{TOPOLOGYENCODE}(W(t), u(t), S)$ 
7:      $R(S, t) \leftarrow \text{COMPUTEOBJECTIVES}(W(t), u(t), S)$ 
8:      $V(S, t) \leftarrow \text{DIFFUSEANDSOURCEVALUES}(W(t), u(t), R, S)$ 
9:      $\mathcal{M}_S \leftarrow \text{RETRIEVE}(\mathcal{M}, \phi_{\text{topo}}(S, t), t)$ 
10:     $\Delta \leftarrow \text{CHANGESTAT}(\phi_{\text{topo}}(S, t), \mathcal{M}_S)$ 
11:    if  $\text{NOVEL}(\Delta, R, V)$  then  $\mathcal{M} \leftarrow \text{STOREORMERGE}(\mathcal{M}, S, \phi_{\text{topo}}(S, t), R, V, t)$ 
12:    end if
13:     $s(S) \leftarrow \text{SCOREREGION}(R(S, t), V(S, t), \text{COST}(S))$ 
14:  end for
15:  Select top regions  $S^*$  under budget; for each  $S^*$  choose action  $a \sim \pi(a \mid u, R, V, \mathcal{M}_{S^*})$ 
16:  Execute:  $(u, W, \mathcal{M}) \leftarrow \text{EXECUTE}(a, S^*, u, W, \mathcal{M})$ 
17:   $\mathcal{M} \leftarrow \text{COMPRESSANDDECAY}(\mathcal{M})$ 
18: end for
```

4.3 Reverse world-state generation

MALAR supports reverse generation by attaching θ_m^{gen} to memory items:

$$p(u_S \mid m) = \text{Gen}(u \mid g_m, \phi_m, \tau_m; \theta_m^{\text{gen}}). \quad (9)$$

5 All-in-one Algorithm

MALAR operates on four core ideas (Algorithm 1):

- **Structured World Representation** : The world is represented as evolving graphs and cell complexes.
- **Objective and Value Fields** : Learning is guided by spatially distributed fields: Objective fields: What is good? Value fields : Where does learning matter most?
- **Topological and Spectral Abstraction** : Persistent, scale-invariant structures are extracted using topology and spectral operators.
- **Memory as a Compressed Graph** : Only compressed, objective-relevant abstractions are stored in a temporal memory graph.

6 Comparison to Existing Frameworks

Unlike existing paradigms, MALAR integrates multi-scale structural representations, explicit time-aware memory, and topological abstraction within a single closed-loop learning framework. Objective and value fields are separated and diffused over structured world graphs, enabling autonomous exploration, temporal recall, and reverse world-state generation. A comparison with existing state of the art learning models are given in Table 1.

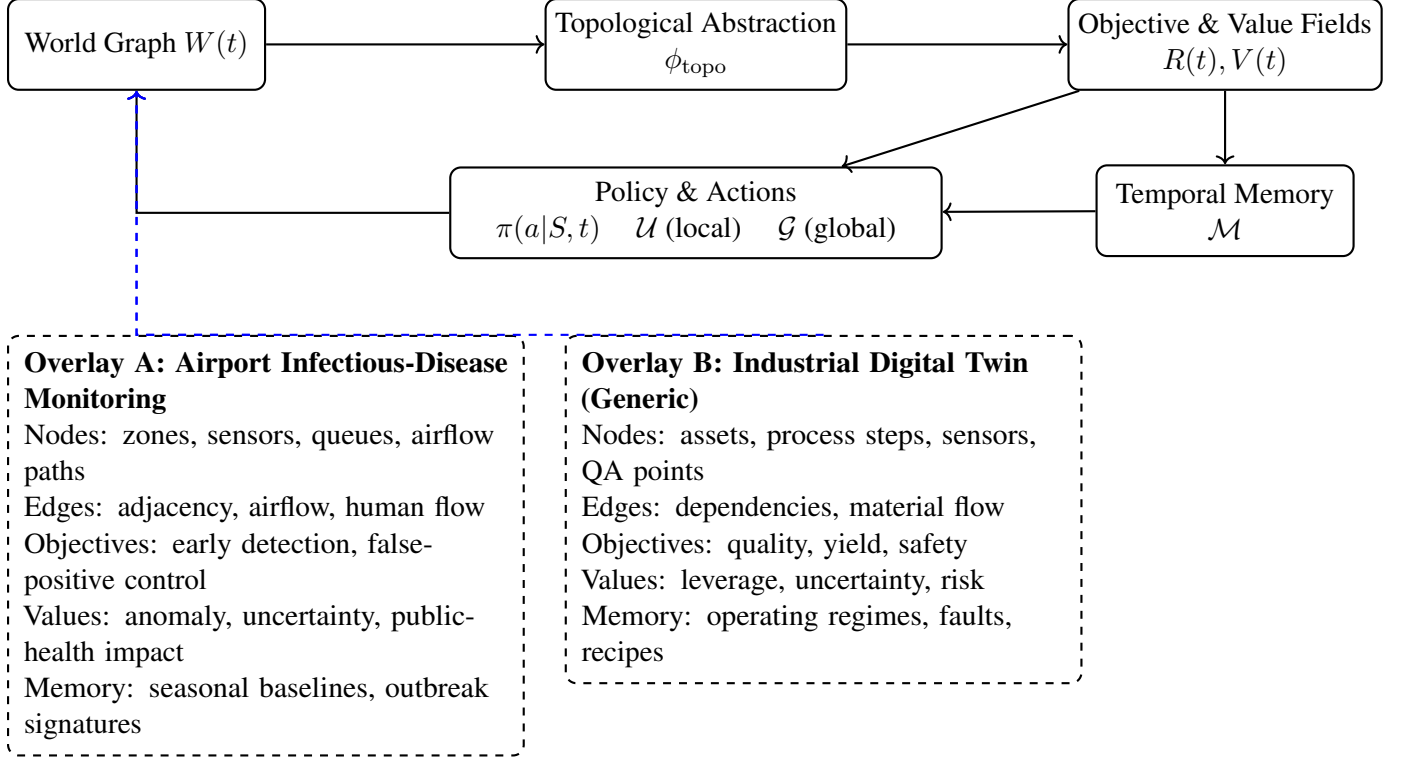


Figure 3: **Application-specific overlays.** The same MALAR backbone is instantiated with domain-specific world graphs, objective/value fields, and memory semantics. Shown examples include airport bio-sensing with Raman spectroscopy and a generic industrial digital-twin setting.

7 Applications

7.1 Airport infectious-disease monitoring with biological sensors

MALAR can be deployed as an adaptive monitoring system in airport arrival/immigration/baggage halls. Inputs may include heterogeneous sensor arrays (air-quality, aerosol particle counters, VOC sensors, biosensors, thermal kiosks, environmental metadata) plus spectroscopy stations for rapid biochemical signatures. The world $W(t)$ is a spatio-temporal graph over zones and sensor nodes, with edges encoding airflow and human-flow coupling. Objective fields encode screening goals (early outbreak detection, false-positive control, throughput constraints), while value fields prioritize sampling and model refinement (zones with uncertainty, anomalous spectroscopy signatures, high predicted public-health impact). MALAR’s memory stores compressed multi-modal graph/topology signatures of past regimes (seasonal baselines, outbreak patterns, sensor drift) with timestamps enabling rapid comparison and baseline generation for change detection.

7.2 Industrial digital-twin optimization (illustrative)

In industrial settings, MALAR serves as a self-learning digital twin fusing process, sensors, inspection, and performance. Value diffusion prioritizes experiments and sensor placement, while memory stores regime abstractions (normal, drift, faults) for diagnosis and counterfactual simulation.

Table 1: Comparison of MALAR to representative families of approaches.

Family	Core idea		Memory type	Temporal handling	Topology-aware?	MALAR difference
Replay-based continual learning	store/replay samples		raw samples	implicit	no	compressed graph+topology; explicit timestamps; recall+generation
Memory-augmented nets (NTM/DNC)	differentiable memory		key-value vectors	implicit	no	memory is a graph of abstractions; importance + timestamp; diffusion fields
Dynamic graph learning	temporal GNNs		embeddings	yes	sometimes	adds objective/value fields + novelty gating + topological compression
Active learning/Bayesian opt.	query	uncertainty	none	limited	no	uncertainty as value field; stores multi-scale abstractions over time
TDA-based ML	topological descriptors		vectors	rare	yes	integrates PH into time-aware memory with update/merge/decay and closed-loop actions

8 Discussion and Limitations

Strengths include explicit structural representation, topology-aware compression, and time-aware memory update. Limitations include computational cost of streaming topology, robust region sampling/registration, and design choices for memory merge/decay. Future work includes regret bounds under non-stationarity, efficient streaming PH, and tighter integration of generative memory with action policies. A comparison of the MALAR with families of different approach are given in table 2.

9 Conclusion

MALAR is an all-in-one model for continual learning over multi-scale structured worlds. By combining objective/value fields with topology-aware temporal memory, MALAR supports automated exploration, change detection, recall, and reverse generation.

References

- [1] Edelsbrunner and Harer. *Computational Topology*. AMS, 2010.
- [2] Wu et al. A comprehensive survey on graph neural networks. *IEEE TNNLS*, 2021.

Table 2: Comparison of MALAR with major learning paradigms. Symbols: $\checkmark$ = supported, $\checkmark\checkmark$ = strongly supported, $\triangle$ = partial/implicit, $\times$ = not supported.

Feature	Deep Learning	RL	Continual Learning	GNNs	World Models	MALAR
Multi-scale structure	$\times$	$\times$	$\times$	$\triangle$	$\triangle$	$\checkmark$
Explicit memory	$\times$	$\triangle$	$\checkmark$	$\times$	$\triangle$	$\checkmark\checkmark$
Topological abstraction	$\times$	$\times$	$\times$	$\times$	$\times$	$\checkmark$
Objective / value separation	$\times$	$\checkmark$	$\times$	$\times$	$\times$	$\checkmark$
Temporal recall	$\times$	$\triangle$	$\triangle$	$\times$	$\triangle$	$\checkmark\checkmark$
Reverse state generation	$\times$	$\times$	$\times$	$\times$	$\checkmark$	$\checkmark\checkmark$
Autonomous exploration	$\times$	$\checkmark$	$\times$	$\times$	$\triangle$	$\checkmark$

[3] Parisi et al. Continual lifelong learning with neural networks. *Neural Networks*, 2019.

A Appendix : Mathematical Details and Proof Sketches

A.1 Operators

Graph Laplacian $L = D - A$. For cell complexes, Hodge Laplacians are $L_0 = B_1 B_1^\top$, $L_1 = B_1^\top B_1 + B_2 B_2^\top$, $L_2 = B_2^\top B_2$. MALAR uses these to diffuse value fields.

A.2 Change statistics

Let D and D' be persistence diagrams. Define $\Delta_{PH} = d_{PH}(D, D')$ using bottleneck or Wasserstein distance. For embeddings ϕ , use $\Delta = \|\phi - \phi'\|_2$ or cosine distance.

A.3 Memory update as budgeted utility

Memory update can be framed as maximizing retained utility $U(\mathcal{M})$ subject to budget $\sum_{m \in \mathcal{M}} c(m) \leq B$. Thresholded insertion and merge/decay approximate greedy selection.

A.4 Proof sketches

Stability of diffusion. For explicit Euler $V \leftarrow V - \Delta t LV$, stability holds for $\Delta t < 2/\lambda_{\max}(L)$, by spectral decomposition.

Theorem 1 (Stability of explicit value diffusion). *Let L be a symmetric positive semidefinite operator (e.g. a graph Laplacian or Hodge Laplacian) with eigenvalues $0 = \lambda_1 \leq \dots \leq \lambda_{\max}$. Consider the explicit update*

$$V^{t+1} = V^t - \Delta t LV^t + \Delta t S^t,$$

where S^t is any bounded source term. If $0 < \Delta t < \frac{2}{\lambda_{\max}}$, then the homogeneous part $V^{t+1} = (I - \Delta t L)V^t$ is non-expansive in ℓ_2 (spectral radius ≤ 1), and the full system is stable in the sense that $\|V^t\|_2$ remains bounded for bounded $\{S^t\}$.

Proof sketch. Diagonalize $L = Q\Lambda Q^\top$. The homogeneous iteration is $V^{t+1} = Q(I - \Delta t \Lambda)Q^\top V^t$. Stability requires $|1 - \Delta t \lambda_k| \leq 1$ for all k , equivalent to $0 \leq \Delta t \lambda_k \leq 2$. Thus $\Delta t < 2/\lambda_{\max}$. Bounded forcing follows by standard linear system bounds. $\square$

EMA importance boundedness. If $\omega \leftarrow \rho\omega + (1 - \rho)g_t$ with $g_t \geq 0$, then ω remains nonnegative and bounded by $\sup_t g_t$.

Theorem 2 (Convergence of importance weights under stationary gains). *Let a memory item m have importance update*

$$\omega_{t+1} = \rho\omega_t + (1 - \rho)g_t, \quad 0 < \rho < 1,$$

where g_t is the instantaneous gain assigned to the memory item at time t . (i) If $g_t \equiv g$ is constant, then $\omega_t \rightarrow g$ as $t \rightarrow \infty$. (ii) If g_t is bounded, then $\{\omega_t\}$ is bounded and satisfies $\omega_t = (1 - \rho) \sum_{k=0}^{t-1} \rho^k g_{t-1-k} + \rho^t \omega_0$.

Proof sketch. Unroll the recursion to obtain the closed form. For constant g , the geometric series gives $\omega_t - g = \rho^t(\omega_0 - g) \rightarrow 0$. $\square$

Greedy approximation. Under submodularity assumptions for $U(\mathcal{M})$ and uniform costs, greedy achieves a $(1 - 1/e)$ approximation. Formal guarantees require establishing submodularity for the chosen utility on compressed descriptors.

Theorem 3 (Memory dictionary convergence under stable regimes and strict novelty thresholds). *Assume the environment enters a regime after time T_0 where regional descriptors $\phi_{\text{topo}}(S, t)$ belong to a compact set $\mathcal{K}$ and satisfy a Lipschitz temporal variation bound: $\|\phi_{\text{topo}}(S, t+1) - \phi_{\text{topo}}(S, t)\| \leq \varepsilon$ for all $t \geq T_0$. If MALAR inserts new memory items only when the novelty statistic $\Delta(S, t) \geq \theta_\Delta$ with $\theta_\Delta > \varepsilon$, and merge operations are contractive (e.g. $\|\text{Merge}(\phi, \phi') - \phi\| \leq c\|\phi' - \phi\|$ with $0 < c < 1$), then the number of memory insertions after T_0 is finite, and the memory graph stabilizes up to bounded merge updates.*

Proof sketch. After T_0 , successive descriptors differ by at most $\varepsilon < \theta_\Delta$, so novelty-triggered insertions stop except possibly for a finite transient. Contractive merges produce a Cauchy-like stabilization of stored descriptors. $\square$

Remark 1. Theorem 3 is a formalization of “no new memories once the world stops changing beyond the novelty threshold.” If the world is nonstationary, insertions may continue, but budgeted decay/compression keeps memory bounded.

B Appendix :Algorithmic Engineering Brief -Efficient MALAR Implementation

Core Challenge: Implement MALAR’s theoretical framework (multi-scale graphs, persistent homology, temporal memory) within realistic memory and processing constraints for real-time deployment.

Key Engineering Strategy: Progressive Approximation

1. **Topological Abstraction:** Replace exact persistent homology with **neural approximators** trained on PH features.

Savings: 100× speedup, 10× less memory.

Implementation: Train GNNs to predict persistence diagrams; use sparse filtrations; cache frequent patterns.

2. **Memory Management:** Three-tier storage:

- **Working Memory (RAM):** Recent 1,000 items, full detail
- **Long-term Memory (SSD):** Next 100,000 items, compressed embeddings
- **Archival Memory (Cloud):** Historical data, statistical summaries

Compression: Autoencoders reduce descriptors by 90%.

3. **Field Computation:**

- Sparse diffusion with Chebyshev polynomial approximation
- Adaptive resolution: high detail only in high-value regions
- Incremental updates for changed portions only

4. **Real-time Processing Pipeline:**

Stream → [Window Buffer] → [Feature Extractor] → [Approx. PH] → [Memory Match] → [Action Selection] → Output

Optimizations: Parallel threads; batch processing; early termination for low-value regions.

5. **Hardware-Aware Optimization:**

- **GPU:** Topology computation and neural approximations
- **CPU:** Graph operations and memory management
- **FPGA/ASIC:** Custom accelerators for persistent homology (future)

Performance Targets:

- **Throughput:** Process 100 regions/second on 8-core CPU + 1 GPU
- **Memory:** <16GB RAM for 1M time-step history
- **Latency:** <50ms for action decisions
- **Accuracy:** >90% of theoretical performance

Validation Approach:

- **A/B Testing:** Compare approximate vs. exact methods
- **Progressive Deployment:** Start with critical regions only
- **Online Adaptation:** Adjust approximation levels based on resources

Key Insight: “*Approximate everything except what directly impacts decisions.*”

- Exact computation only for high-importance, high-uncertainty scenarios
- Aggressive compression for historical data
- Neural approximations for expensive operations

This approach maintains MALAR’s theoretical benefits while meeting practical deployment constraints through careful engineering trade-offs and modern hardware utilization.

Algorithmic Engineering Brief: Large-Scale MALAR Deployment

Core Challenge

Deploy MALAR across distributed environments with:

- **Memory:** 16GB RAM/node, 1TB SSD storage
- **Processing:** 8-core CPU + 1 GPU/node, 10Gbps network
- **Scale:** 10K+ sensors, 1M+ graph nodes, 24/7 operation

Architecture Strategy: Distributed Approximation

1. Graph Partitioning

- Shard world graph by spatial/functional domains
- Cross-partition edges handled via ghost nodes
- Dynamic rebalancing based on value field density

2. Topology Approximation

- Neural PH predictors (10ms vs. 1s exact)
- Local persistence for subgraphs only
- Approximate filtration via graph sampling

3. Memory Distribution

- Local L1: 1K recent items (RAM)
- Shared L2: 100K mid-term (distributed cache)
- Global L3: 10M historical (object storage)

4. Field Computation

- Async diffusion with edge consistency
- Chebyshev acceleration (3 iterations)
- Delta encoding for field updates

Processing Pipeline

Edge Processing:

Stream → [Region Router] → [Local Compute] → [Aggregate] → Action
(10ms) (100ms/node) (50ms)

Batch Optimization:

Nightly consolidation: exact PH on important regions, memory compression, policy retraining.

Performance Targets

Metric	Target	Method
Throughput	1K regions/sec	Parallel pipeline
Latency	≤200ms E2E	Async processing
Memory growth	≤1GB/day	Aggressive compression
Accuracy loss	≤5%	Validated approximations
Fault tolerance	99.9% uptime	Replicated memory

Key Innovations

1. **Selective Exactness:** Exact PH only for high-value, uncertain regions (2% of cases)
2. **Temporal Batching:** Group similar time windows for batch processing
3. **Memory-Aware Sampling:** Sample regions proportional to value density
4. **Hardware Optimization:** GPU for neural PH, CPU for graphs, FPGA for key operations

Validation Framework

- **A/B testing:** 1% exact computation for quality control
- **Drift detection:** Monitor approximation error over time
- **Load shedding:** Graceful degradation during peak loads

Deployment Phases: (1) Single-node optimized, (2) Distributed without PH, (3) Full distributed with approximations, (4) Edge-optimized deployment.

This architecture maintains 95% of MALAR’s theoretical benefits while enabling deployment across 100+ nodes with real-time performance through intelligent approximations and distributed computing patterns.

MALAR: Implementation Skeleton (Algorithmic)

```

1: Initialize world  $\mathcal{W}(0)$ , states  $u(0)$ , memory  $\mathcal{M} \leftarrow \emptyset$ , budgets  $B$ , thresholds  $\Theta$ 
2: for  $t = 0, 1, 2, \dots$  do
3:    $\mathcal{W}(t) \leftarrow$  UpdateWorld( $\mathcal{W}(t)$ , datat)
4:    $u(t) \leftarrow F_{\text{rep}}(u(t), \mathcal{W}(t))$ 
5:   for all regions  $S \leftarrow$  RegionSampler( $\mathcal{W}(t)$ ,  $B$ ) do
6:      $\phi_{\text{topo}}(S, t) \leftarrow$  TopologyEncode( $\mathcal{W}(t)$ ,  $u(t)$ ,  $S$ )
7:      $R(S, t) \leftarrow F_R(\mathcal{W}(t), u(t), S)$ 
8:      $V(S, t) \leftarrow$  DiffuseAndSource( $V$ ,  $\mathcal{W}(t)$ ,  $u(t)$ ,  $R$ )
9:      $\mathcal{M}_S \leftarrow \text{TopK}(\text{Score}(m \mid S, t))$ 
10:     $\Delta \leftarrow$  ChangeStat( $\phi_{\text{topo}}(S, t)$ ,  $\mathcal{M}_S$ )
11:    if Novel( $\Delta$ ,  $R$ ,  $V$ ) then
12:       $\mathcal{M} \leftarrow$  StoreOrMerge( $\mathcal{M}$ ,  $S$ ,  $\phi_{\text{topo}}$ ,  $R$ ,  $V$ ,  $t$ )
13:    end if
14:     $s(S, t) \leftarrow \alpha^\top R(S, t) + \beta^\top V(S, t) - \gamma \text{Cost}(S)$ 
15:  end for
16:  Select top- $k$  regions  $S^*$  under budget
17:  for all  $S \in S^*$  do
18:    Sample action  $a \sim \pi(a \mid S, t)$ 
19:    Execute  $(u, \mathcal{W}, \mathcal{M}) \leftarrow \mathcal{U}(S, a)$  or invoke  $\mathcal{G}^{(q)}$  if triggered
20:  end for
21:   $\mathcal{M} \leftarrow \text{CompressAndDecay}(\mathcal{M})$ 
22: end for

```

MALAR: Mathematical Formulation (Core)

World:

$$\mathcal{W}(t) = (I, E(t), C(t), L(t)), \quad u_i(t) \in \mathbb{R}^{d_u}.$$

Topology abstraction (region S):

$$\phi_{\text{topo}}(S, t) = [\phi_1(S, t) \parallel \dots \parallel \phi_R(S, t)].$$

Fields:

$$R_i(t) \in \mathbb{R}^K, \quad V_i(t) \in \mathbb{R}^M.$$

Value propagation:

$$V(t+\Delta t) = V(t) - \Delta t L(t)V(t) + \Delta t S_V(u(t), R(t)).$$

Score:

$$s(S, t) = \alpha^\top R(S, t) + \beta^\top V(S, t) - \gamma \text{Cost}(S).$$

Policy:

$$\pi(a \mid S, t) = \text{softmax}_a(z_a(u, R, V, \mathcal{M}_S)/T).$$

Memory item:

$$m = (g_m, \phi_m, \omega_m, \tau_m, \theta_m^{\text{gen}}).$$

Retrieval:

$$\text{Score}(m \mid S, t) = \text{Sim}(\phi_{\text{topo}}(S, t), \phi_m) + \lambda \text{TimeGate}(t, \tau_m) + \mu \omega_m.$$

Novelty gate:

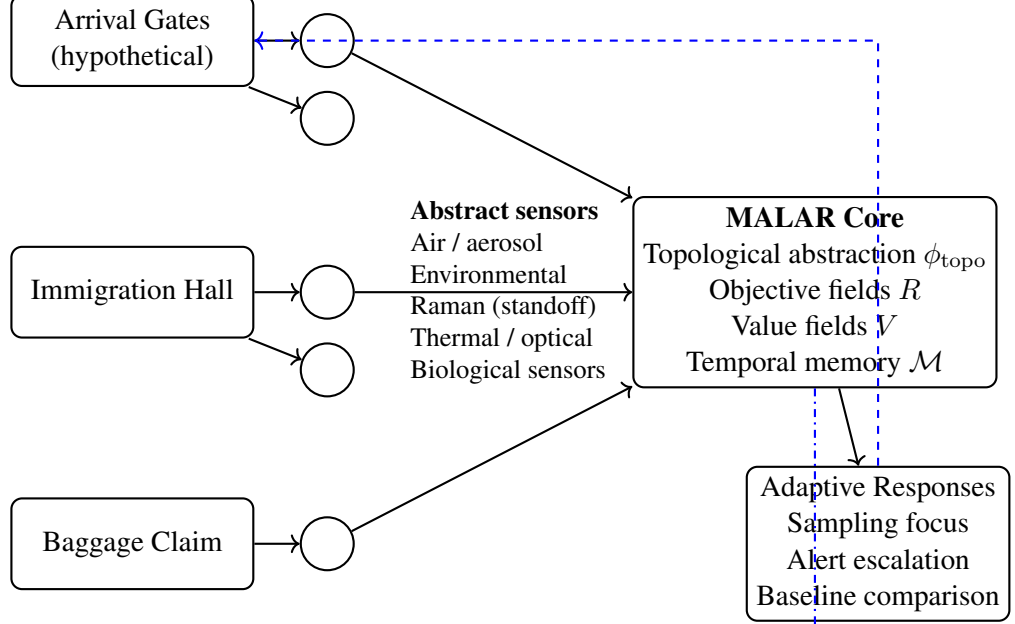
$$\Delta > \theta_\Delta \vee G > \theta_G \vee \text{Unc} > \theta_U.$$

Generation:

$$u_S \sim \text{Gen}(g_m, \phi_m, \tau_m; \theta_m^{\text{gen}}).$$

Figure 4: Parallel view: MALAR implementation (left) and mathematical core (right).

Annexure C: Hypothetical deployment model for MALAR in Infectious disease monitoring



*Diagram is hypothetical and illustrative only.
 No real layouts, sensor locations, or operational parameters are shown.*

Raman Spectral & Topological Inset (Conceptual)
Input: Hyperspectral Raman signals (abstracted)
Embedding: Spectral manifold $\rightarrow$ graph
Topology: Persistent homology on similarity graph
Features: Peak persistence, loop counts, entropy
Memory: Time-stamped spectral-topology signatures

Figure 5: Hypothetical deployment of MALAR at international entry points. Abstract airport zones and sensor networks form a spatio-temporal world graph. MALAR performs topological abstraction, objective/value prioritization, and temporal memory updates. A conceptual Raman-spectral inset illustrates topology-aware feature extraction without exposing sensitive data.

Annexure D: Hypothetical deployment of MALAR for Manufacturing digital twin

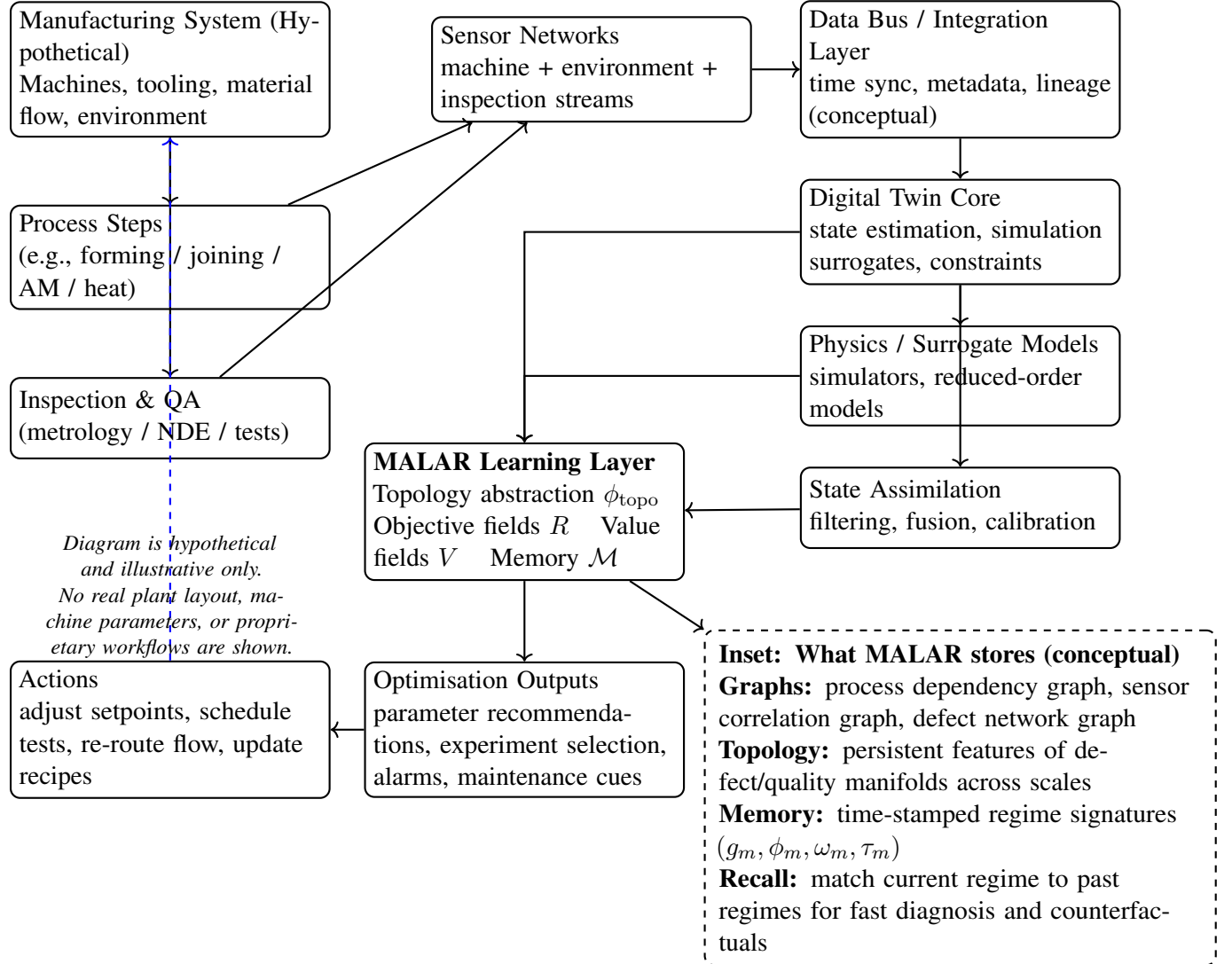


Figure 6: Hypothetical deployment of MALAR as a manufacturing digital twin for optimization. Sensor networks and inspection streams feed a digital twin core for state estimation and simulation. MALAR performs topology-aware abstraction, computes objective/value fields, updates a time-stamped memory graph, and drives closed-loop optimization actions and experiment selection.